Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Chapter 3 - NoSQL Data Management

Study Guide

Ms. Janhvi Dave
Assistant Professor, CSE
Parul Institute of Technology

IN

1. What is NoSQL?

- The term NoSQL stands for “Not Only SQL” (or “non-relational”) database systems.
- These are database management systems that depart from the conventional relational (table-based) model. They are designed to handle large volumes of **structured, semi-structured and unstructured data**, with flexible schema, and often with distributed architecture.
- Key characteristics:
    - Flexible / schema-less (or schema-on-read) data models.
    - Horizontal scalability (scale-out) vs traditional vertical scaling.
    - Often support for distributed storage, high availability, partitioning/sharding.
    - Trade-offs in consistency models (CAP/BASE) vs ACID in relational systems.

Why NoSQL?

- Modern applications generate enormous data volumes (Big Data) with variety (text, JSON, images), velocity (streams), and need agile iteration and scaling, which relational DBs may struggle to support.
- Relational DBs require rigid schema changes for evolving data, and often scale vertically (adding resources to single node) which becomes expensive. NoSQL supports more agile development and cloud/distributed scaling.
- Use-cases: Web & mobile apps, real-time analytics, IoT, social networks, large user-scale platforms.

Advantages of NoSQL (High level)

- Flexible data models → adapt quickly.
- High scalability and performance for large, distributed datasets.
- Suitable for large amounts of semi-structured/unstructured data.
- Support for agile development (less time managing schema, more focusing on features).

Disadvantages / Considerations

- Maturity: Some NoSQL systems are newer and may lack the tooling, ecosystem, expertise compared to relational systems.
- Consistency trade-offs: Many NoSQL systems sacrifice strict ACID transactional guarantees to achieve higher availability and partition tolerance (BASE instead of ACID).

- Sometimes difficult queries / joins or transactional semantics compared to relational DBs.
- Choosing the right NoSQL type and managing distributed complexity (sharding, replication, consistency) adds operational overhead.

Table: Intro summary

Aspect	Relational (SQL)	NoSQL
Schema	Fixed, predefined	Flexible / dynamic
Data model	Tables, rows, columns	Documents, key-value, graph, etc.
Scalability	Vertical (scale up)	Horizontal (scale out)
Consistency model	ACID	Often BASE / eventual consistency
Use-cases	Structured, transactional	Big Data, semi/unstructured, Web

2. Types of NoSQL

NoSQL databases are categorized into four main types — **Key-Value Stores, Document Stores, Wide-Column Stores,** and **Graph Databases** — each designed to handle specific kinds of data and workloads efficiently.

1. **Key-Value Stores** store data as pairs of unique keys and associated values, enabling extremely fast lookups and simple scalability through partitioning by key. They are best suited for use cases such as caching, session storage, and user profile management where quick retrieval by key is critical. However, they lack complex querying and joining capabilities since the values are typically opaque.
   Example: Redis, Riak, Amazon DynamoDB.

2. **Document Stores** manage semi-structured data in flexible document formats such as JSON or BSON, where each document can have a unique structure. They support nested data, indexing, and rich querying, making them ideal for applications like e-commerce product catalogs, content management systems, and real-time analytics. Though highly flexible, they can face challenges with multi-document transactions and deep relational queries.
   Example: MongoDB, CouchDB, Firebase Firestore.

3. **Wide-Column Stores** organize data into column families instead of rows, allowing each record to have variable columns. This model is optimized for massive scalability, high write throughput, and analytical workloads such as sensor data storage, time-series analytics, or large-scale event tracking. While extremely powerful for handling large datasets, they can be complex to design and maintain.
   Example: Apache Cassandra, HBase, ScyllaDB.

4. **Graph Databases** represent data as nodes (entities) and edges (relationships), both of which can have properties. This model is particularly effective for scenarios involving complex relationships, such as social networks, fraud detection, recommendation systems, and network topology management.
   They enable efficient traversal of connections but can be harder to scale for very large graph datasets.

Example: Neo4j, Amazon Neptune, ArangoDB.

Summary Table

Type	Description & Use-Case
Key-Value Store	**Store pairs of key: value. Very simple model. Excellent for caching, session stores, shopping carts.**
Document Store	**Data stored as documents (e.g., JSON, BSON) with nested structure. Flexible and developer friendly.**
Wide-Column / Column-Family Store	**Data stored in column families (not strictly rows), optimized for queries over many columns, large scale analytics.**
Graph Database	**Data stored as nodes and edges (relationships). Ideal for social networks, recommendation engines, fraud detection.**
***(Sometimes)* In-Memory / Time-Series etc.**	**Some overviews include in-memory NoSQL or time-series NoSQL as additional categories.**

3. Why NoSQL?

Key Advantages

- **Scalability**: NoSQL systems are designed to scale out horizontally across many commodity servers (nodes) rather than relying on a single large server.
- **Flexibility of schema**: They allow data models to evolve without the heavy migrations of relational DBs.
- **Handling big data**: Suitable for very large datasets (volume), high velocity writes/reads, and variety of data types.
- **Agile development**: Since schema is flexible and many NoSQL systems are developer-friendly, iteration can be faster.

- **High availability & distributed nature**: Many NoSQL systems support replication, partitioning, and resilience across nodes/regions.

Key Disadvantages

- **Consistency/transaction trade-offs**: Many sacrifice strong consistency or full ACID transactional support for scalability and availability (the CAP theorem).
- **Less mature tooling**: Compared to decades of relational database ecosystem, some NoSQL systems may have fewer tools, less proven for certain enterprise workloads.
- **Complex operations**: Managing distributed systems, sharding, replication, eventual consistency, failover — requires more careful engineering.
- **Query limitations**: Complex joins, ad-hoc relational queries, multi-document transactions may be less efficient or not supported.
- **Not for all use-cases**: If your data is highly relational, schema stable, and requires strong transactional guarantees, a relational DB may still be better.

When to choose NoSQL

- If you expect rapid growth of data size or wide distribution across data centers.
- If your data is semi/unstructured, schema may change over time.
- If you require very fast read/write at scale, and maybe eventual consistency is acceptable.
- If your application aligns with the data model (key-value, document, graph) rather than rigid tabular relations.

4. Comparison: SQL vs NoSQL vs NewSQL

What is NewSQL?

NewSQL refers to a class of **modern relational database systems** designed to bridge the gap between traditional SQL and NoSQL databases. These systems retain the **relational data model and ACID (Atomicity, Consistency, Isolation, Durability) properties** of conventional databases, ensuring data reliability and transactional integrity.

At the same time, **NewSQL databases** incorporate the **horizontal scalability and distributed architecture** typically associated with NoSQL systems. This allows them to handle **high-performance, large-scale**

workloads without sacrificing data consistency. In essence, NewSQL combines the **robust consistency of SQL** with the **scalability and flexibility of NoSQL**, making it ideal for modern, data-intensive applications.

Comparison Table

Feature	SQL (Relational)	NoSQL	NewSQL
Data model	Tables/rows/columns, fixed schema	Flexible models (documents, key-value, graph)	Relational model (tables) with modern engine
Schema	Predefined, strict	Dynamic/flexible	Like SQL – schema, but more flexible
Scalability	Vertical (scale up)	Horizontal (scale out)	Horizontal + relational features
Transaction support	Strong ACID	Often weaker consistency (BASE)	ACID + scalability
Best suited for	Structured data, complex relations, stable schema	Big data, semi/unstructured data, rapid changes	High throughput OLTP + scalability
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis	VoltDB, Google Spanner, NuoDB

Summary of differences

- **SQL:** mature, stable, relational, strong consistency, but may struggle with extreme scale or schema flexibility.
- **NoSQL:** very scalable, flexible, suited for big data and evolving schemas—but may trade off consistency or relational features.
- **NewSQL:** tries to get best of both—relational features + large scale. Still comparatively newer.

Advantages/Disadvantages Summary

- SQL advantages: rich query language, transactional integrity, mature ecosystem. Disadvantages: scaling can be harder/costly, schema changes can be disruptive.

- NoSQL advantages: scale, flexibility, suited for unstructured data. Disadvantages: less transactional / relational support, less mature for some use-cases.
- NewSQL: potential to deliver both, but fewer deployments, newer, potential vendor lock-in or operational complexity.

5. Aggregates, Key-Value & Document Data Models

Aggregates

- In NoSQL literature, especially in document stores, the concept of an **aggregate** (from Domain-Driven Design) is used: a cluster of domain objects that can be treated as a single unit for data changes.
- Rather than many small normalized tables (as in relational), NoSQL models often favour storing an aggregate or document which encapsulates related data together.
- This helps reduce joins, improves locality of data access, and aligns with how many applications use data.

Key-Value Data Model

- Very simple: each item is stored as a unique key and an associated value (which might be a blob, JSON, etc.).
- The system essentially acts like a giant hash table.
- Pros: extremely fast for lookup by key, trivially scalable.
- Cons: limited querying by other attributes, values often opaque to the store, harder to express complex relationships.

Document Data Model

- Documents (JSON, BSON) store structured/semi-structured data. Fields can be nested, arrays allowed.
- A collection of documents forms the store. Documents need not have identical schema.
- Pros: aligns closely with application object model, flexible schema, good query support on document fields, nested data accessible.

- Cons: if you have many cross-document relations, then either you embed everything (which may cause duplication) or you must manage references manually; may lead to complexity when modelling deep relations.

How aggregates relate to these models

- In document stores, you typically design the document to reflect an aggregate: for example a "Customer" document may include embedded "orders" or order summaries, rather than a separate orders table and join.
- The aggregate boundary is important: you want to include data that is accessed/updated together in the same aggregate/document, to minimize cross-document joins and distributed transactions.

6. Graph Databases

What is a Graph Database?

- Graph databases store data as **nodes** (entities) and **edges** (relationships) with properties on each. The relationships themselves are first-class and can be traversed very efficiently. Google Cloud+1
- Typical use-cases: social networks (friends, followers), recommendation engines (users/items, relationships), fraud detection (complex relationships), network/IT infrastructure modelling.

Key Features

- Schema may still exist, but the emphasis is on flexible relationships.
- Traversal queries (e.g., "friends of friends", path finding) are more efficient than in table-based models.
- Graph model reduces the impedance of mapping relational tables into graph-like relationships.

Advantages

- Natural fit for relationship-centric data models.
- Efficient for graph algorithms, traversals, deeply connected data.
- Flexibility in evolving relationships, adding new types of nodes/edges.

Disadvantages

- Usually not optimized for massive analytical workloads (though some do).
- Scaling (especially horizontal) may be more complex than simple key/value stores.
- Less familiarity/less tooling for some developers compared to relational or document stores.

7. Map-Reduce, Partitioning and Combining

Map-Reduce in Big Data & NoSQL context

- Hadoop MapReduce is a programming model for processing large data sets in a distributed, parallel fashion: a "map" phase, a "shuffle/partition" phase, and a "reduce" phase.
- It is relevant to NoSQL/Big Data because many NoSQL systems either integrate with or support map-reduce–style processing over distributed data, especially in analytics contexts.

Partitioning & Combining

- **Partitioning (Sharding)**: Splitting data across nodes, often by key or hash, so that different parts of the data reside on different nodes (thus improving parallelism and scale).
- **Combining**: An optimization step often used between Map and Reduce to pre-aggregate or pre-combine results locally before sending across network, thereby reducing data shuffle overhead.

Table: Map-Reduce/Partitioning concepts

Concept	Description	Impact on NoSQL / Big Data
Map	Apply a function to each data item(s), produce intermediate key/value pairs	Useful for distributed transformations or filtering
Shuffle/Partition	Redistribute intermediate data by key to reducers	Data locality important; partition key selection matters
Reduce	Aggregate/group intermediate data per key into final results	Enables summarization, grouping, aggregation
Combiner	Local reduction before shuffle to reduce data volume	Improves performance by reducing network traffic

Sharding/Partitioning	Horizontal splitting of data across nodes	Key design decision for NoSQL data layout & scale

Summary & Key Take-aways

- NoSQL is not a single technology but a family of database systems designed for scale, flexibility and modern data requirements.
- Choosing the right NoSQL type (key-value, document, column, graph) depends on your data model and access patterns.
- While NoSQL offers many benefits (scalability, flexibility), it also brings trade-offs (consistency, operational complexity).
- Understanding relational (SQL) vs NoSQL vs NewSQL helps you make informed choices: matching technology to application requirements.
- Big Data analytics often involve map-reduce, partitioning, combining, and sharding – all of which link closely to how NoSQL systems are architected and used.

References:

1. NoSQL Database. Wikipedia. Retrieved from https://en.wikipedia.org/wiki/NoSQL

2. MongoDB Documentation. Overview of Document Databases. Retrieved from https://www.mongodb.com/nosql-explained

3. Cassandra Documentation. Wide-Column Store Concepts. Retrieved from https://cassandra.apache.org/_/index.html

4. Redis Documentation. Introduction to Key-Value Databases. Retrieved from https://redis.io/docs/about/

5. Neo4j Graph Database Platform. Graph Data Model and Use Cases. Retrieved from https://neo4j.com/developer/graph-database/

6. Google Cloud. What is NoSQL? Retrieved from https://cloud.google.com/discover/what-is-nosql

7. XenonStack. SQL vs NoSQL vs NewSQL Explained. Retrieved from https://www.xenonstack.com/blog/sql-vs-nosql-vs-newsql

8. IBM Developer. Introduction to MapReduce and Big Data Processing. Retrieved from https://developer.ibm.com/articles/mapreduce/

9. GeeksforGeeks. Introduction to NoSQL and Its Types. Retrieved from https://www.geeksforgeeks.org/introduction-to-nosql/

10. AWS Documentation. Overview of NoSQL Databases on AWS. Retrieved from https://aws.amazon.com/nosql/

Parul® University
Vadodara, Gujarat
NAAC GRADE A++